

FlowHub

Ein KI-gestützter persönlicher Eingangskorb

Andreas Imboden · CAS AI-Assisted Software Engineering · FFHS FS26

Das Projekt — und die Erfahrung, es mit KI zu bauen



Das Problem: Capture without friction

Der digitale Alltag produziert ständig kleine Informationsschnipsel:
ein Film den man schauen will, ein Artikel zum Lesen, das Foto einer Quittung.

Heute landen sie überall — oder werden vergessen:

Idee → Welche App? → App öffnen → Kategorisieren → Ablegen
= 5+ Schritte, Kontextwechsel, oft vergessen

Der Nutzer will festhalten, ohne im Moment zu entscheiden, **wohin** es gehört.

Die Idee: ein Eingang, KI macht den Rest

Heute

Idee → welche App? → öffnen →
kategorisieren → ablegen

5+ Schritte

Ein **Telegram-Bot** als einziger Eingang.

FlowHub **erkennt** den Input, **kategorisiert** ihn und **routet** ihn
automatisch an den richtigen Self-Hosted-Service im eigenen Homelab.

Mit FlowHub

Idee → **Telegram** → fertig

1 Schritt — KI übernimmt

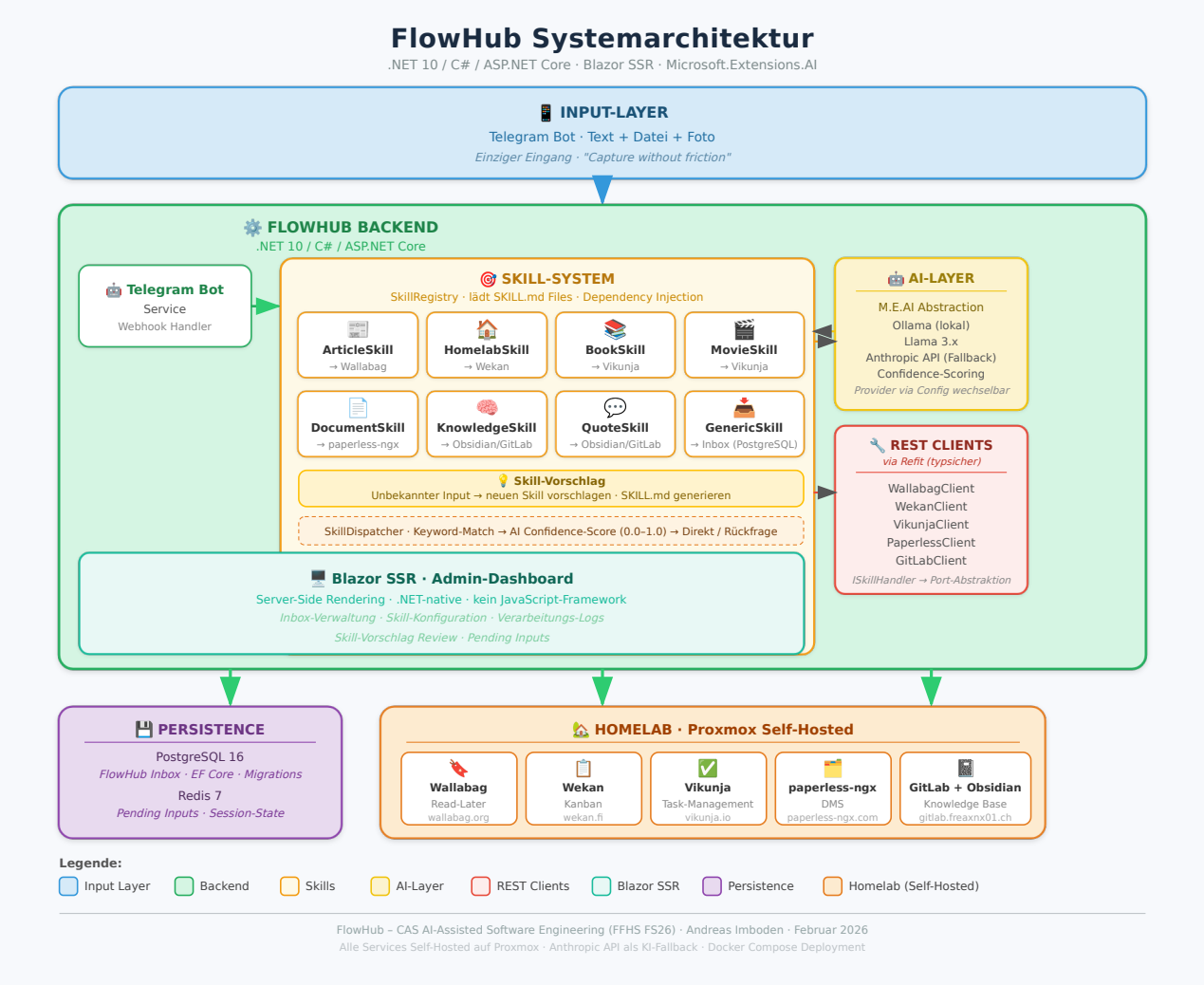
Konkret: das Skill-System

Jeder Input wird einem **Skill** zugewiesen — Erkennung über Keywords, URL-Muster und, wenn nötig, ein **lokales LLM**.

Input	Skill	landet in
heise.de-Artikel	ArticleSkill	Wallabag (read-later)
„The Imitation Game“, share.google/...	MovieSkill	Vikunja (Watchlist)
jellyfin.org „ausprobieren“	HomelabSkill	Wekan (Kanban)
Foto einer Quittung	DocumentSkill	paperless-ngx (DMS)
nichts passt	GenericSkill	Inbox (PostgreSQL)

Unklar? → Der Bot **fragt mit 2-3 Optionen zurück** (Confidence-Score).

Architektur



Tech-Stack

Schicht	Technologie
Backend	.NET 10 / C# / ASP.NET Core (LTS, Nov 2025)
Web-UI	Blazor SSR — .NET-native, kein JS-Framework
KI-Integration	Microsoft.Extensions.AI + Ollama (lokal) / Anthropic (Fallback)
Pipeline	MassTransit — In-Memory (dev) / RabbitMQ (prod)
Persistenz	PostgreSQL 17 + pgvector (semantische Suche) · EF Core 10
REST-Clients	Refit — typsicher, deklarativ
Deployment	Docker Compose — 6 Services, Migrations als Init-Container

Inkrementell über 5 Blöcke gebaut: Konzept → UI → Services/KI → Persistenz → Deployment.

Betrieb: beobachten & selbst heilen

Was	Wie
Metriken	OpenTelemetry → Prometheus (<code>/metrics</code>) → Grafana
Health	<code>/health/live</code> · In-App-Integration-Health (Vikunja/Wallabag/Paperless)
Demo-Status	öffentliche Uptime-Kuma -Statusseite (<code>status.demo.flowhub...</code>) — prüft auch LLM-Erreichbarkeit
Self-Healing	Container- <code>restart</code> -Policies + Healthchecks · KeywordClassifier-Fallback bei LLM-Ausfall

Block-5-Lernziel „Systeme überwachen und optimieren“ — von der App bis zur Demo.

Teil 2: Bauen mit KI

Wie war es wirklich?



Nicht „Prompt rein, Code raus“ — eine Pipeline

Jeder grössere Baustein lief durch denselben strukturierten Ablauf:

Brainstorm → Spec → Plan → Subagent-Implementierung → 2-stufiges Review

1. **Brainstorming-Skill** — Design als A/B/C-Entscheidungen (z. B. 13 Entscheide für die Async-Pipeline), jede mit Begründung festgehalten
2. **Spec + Plan** — schriftliches Design, dann TDD-geordneter Aufgabenplan
3. **Subagenten** — pro Task ein frischer Implementierer (Test-First)
4. **Review ×2** — Spec-Konformität, dann Code-Qualität — bevor etwas in `main` geht

KI-Anteil in Zahlen (Block 4: Persistenz)

Artefakt	Zeilen	KI-generiert	Mensch	KI %
Entity-Klassen + Configs (14)	~250	~225	~25	90 %
Repository-Implementierungen (6)	~350	~315	~35	90 %
Integrationstests (16)	~230	~210	~20	91 %
Docker Compose	~50	~40	~10	80 %
... Services · Filter · Refactor	~130	~112	~18	86 %
Gesamt	~1010	~902	~108	~89 %

Über alle Blöcke: **~85-95 % des Codes KI-generiert.**

Der Mensch-Anteil ist klein — aber **hochwertig.**

Wo die KI glänzt

Boilerplate

7 strukturgleiche `IEntityTypeConfiguration<T>`-Klassen — komplett generiert.
Von Hand zeitintensiv und fehleranfällig.

Migrations & Infrastruktur

EF-Core-Migrations, Refit-Interfaces, GitHub-Actions-YAML — Muster auf Anhieb korrekt.

Tests

16 Integrationstests gegen echtes PostgreSQL (Testcontainers) —
alle grün beim ersten Durchlauf. Konsistente Arrange/Act/Assert-Struktur.

| *KI als **Accelerator**: was repetitiv und gut spezifiziert ist, entsteht in Minuten.*

Wo die KI scheitert

Genau dort, wo **Domänenverständnis** oder **Performance-Gespür** nötig ist:

- **N+1-Blindheit** — `ListAsync` ohne `.Include(c => c.Tags)`. Kein implizites Performance-Bewusstsein für Navigation Properties.
- **CASCADE überall** — alle Fremdschlüssel auf `CASCADE DELETE`. Die Unterscheidung owned vs. referenced (Soft-FK für Audit-Trail) war eine **menschliche Domänen-Entscheidung**.
- **Feldlängen** — `varchar(128)` statt `(64)`. KI weicht ohne expliziten Spec-Verweis auf „sichere“ Defaults aus.
- **Veraltete Paket-Versionen** — Plan pinnte Npgsql 9; real war 10 nötig.
Trainingsdaten hinken neuen Releases hinterher.
- **Feature-Drift** — KI schlägt ständig mehr Features vor; bewusste MVP-Eingrenzung war nötig.

Der Smoke-Test-Moment

Die KI schrieb den ganzen Deployment-Stack. Dann lief **ein** Befehl:

`make smoke-prod` — End-to-End-Probe des laufenden Stacks.

An einem Nachmittag fand er 5 reale, latente Bugs:

- `.editorconfig` nicht ins Build-Image kopiert → Build bricht mit `TreatWarningsAsErrors` ab
- Compose-Env-Casing `${EMBEDDINGS__APIKEY}` $\neq$ `Embeddings__ApiKey` → Embeddings still no-op
- Leerstring-Modellname triggert `AssertNotNullOrEmpty` → Crash beim Start
- Mistral lehnt das `dimensions`-Feld ab → 422
- Passbolt-Refs vom Makefile überschattet → KI-Call erreichte nie den Provider

*KI schrieb den Code. Eine **KI-gestützte Prüfung** fand, was die KI übersah.*

Der Mensch bleibt im Loop — als Reviewer.

Fazit

KI ist ein starker Accelerator für Infrastruktur-Code —

Boilerplate, Migrations, Tests entstehen in Minuten.

Sie braucht menschliche Führung bei Architektur & Domäne —

FK-Strategie, Performance, Scope, aktuelle Versionen.

Der Mensch bleibt Architekt und Reviewer.

Code & Doku: github.com/freaxnx01/FlowHub-CAS-AISE · Danke — Fragen?